

1/22/2025	9 AM - 10 AM
-----------	--------------

Trigger:

DML Trigger

Update, Delete/Insert

DML Trigger in SQL

A **DML (Data Manipulation Language)** trigger in SQL is a type of trigger that is activated by DML operations, which include the INSERT, UPDATE, and DELETE operations on a table or view. These triggers allow you to automate actions when data is modified in a database, such as enforcing business rules, auditing changes, or maintaining data integrity.

Types of DML Triggers:

1. **INSERT Trigger:** Activated when a new record is inserted into a table.
2. **UPDATE Trigger:** Activated when an existing record is modified.
3. **DELETE Trigger:** Activated when a record is deleted.

DML triggers can be created with different timings:

- **BEFORE Trigger:** Executes before the DML operation (e.g., before an INSERT or UPDATE).
- **AFTER Trigger:** Executes after the DML operation (e.g., after an INSERT, UPDATE, or DELETE).
- **INSTEAD OF Trigger:** Executes instead of the DML operation (mostly used with views).

Syntax for Creating DML Triggers:

sql

```
CREATE TRIGGER trigger_name
```

```
[BEFORE | AFTER | INSTEAD OF] INSERT | UPDATE | DELETE
```

```
ON table_name
FOR EACH ROW
BEGIN
    -- Trigger actions (SQL statements)
END;
```

Example Scenarios for DML Triggers:

1. INSERT Trigger Example:

You might want to log every insertion of a new record into an employees table into an audit_log table.

sql

```
CREATE TRIGGER log_employee_insert
AFTER INSERT
ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (action, table_name, record_id, timestamp)
    VALUES ('INSERT', 'employees', NEW.employee_id, NOW());
END;
```

- **AFTER INSERT:** This trigger executes after a new record is inserted into the employees table.
- **NEW.employee_id:** This refers to the employee_id value of the newly inserted row.
- **audit_log:** The action is logged in the audit_log table with the type of action (INSERT), the table name (employees), the employee_id, and the timestamp.

2. UPDATE Trigger Example:

Let's create a trigger that enforces a rule on salary updates to ensure that the salary cannot be reduced below a minimum threshold.

sql

```
CREATE TRIGGER check_salary_update
BEFORE UPDATE
ON employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 30000 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary cannot be less than 30,000';
    END IF;
END;
```

- **BEFORE UPDATE:** This trigger fires before an update operation on the employees table.
- **NEW.salary:** Refers to the updated value of the salary column.
- **SIGNAL SQLSTATE '45000':** Raises an error if the new salary is less than 30,000, preventing the update.

3. DELETE Trigger Example:

This trigger prevents the deletion of an order if there are associated records in the order_details table.

sql

```
CREATE TRIGGER prevent_delete_if_order_has_details
BEFORE DELETE
ON orders
FOR EACH ROW
BEGIN
    IF EXISTS (SELECT 1 FROM order_details WHERE order_id = OLD.order_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Cannot delete an order with related details';
    END IF;
```

END;

- **BEFORE DELETE:** The trigger fires before a delete operation on the orders table.
- **OLD.order_id:** Refers to the order_id of the record being deleted.
- **SIGNAL SQLSTATE '45000':** If related records exist in the order_details table, the deletion is blocked with a custom error message.

4. INSTEAD OF Trigger Example (for Views):

Sometimes, DML triggers are used with views. If you try to perform an INSERT or UPDATE on a view, the trigger can handle the action instead of the database executing the operation directly on the underlying tables.

sql

```
CREATE TRIGGER instead_of_insert_view
```

```
INSTEAD OF INSERT
```

```
ON my_view
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO my_table (col1, col2)
```

```
    VALUES (NEW.col1, NEW.col2);
```

```
END;
```

- **INSTEAD OF INSERT:** This trigger executes instead of performing the INSERT operation directly on the view my_view.
- **NEW.col1, NEW.col2:** These refer to the values you want to insert into the base table my_table.

Key Points for DML Triggers:

- **NEW:** Refers to the values in the row that is being inserted or updated (used in INSERT and UPDATE triggers).
- **OLD:** Refers to the values in the row that is being deleted or updated (used in DELETE and UPDATE triggers).
- **SIGNAL SQLSTATE:** Used to raise custom errors when a condition is met in BEFORE triggers to block the DML operation.

Advantages of DML Triggers:

- **Automation:** Automatically perform actions like logging, validation, and enforcement of business rules.
- **Data Integrity:** Ensure that the data is valid and consistent when modified.
- **Auditing:** Track changes in data for security or historical purposes.

Disadvantages:

- **Performance Impact:** Triggers can add overhead to DML operations, particularly if there are many triggers or complex logic.
- **Complexity:** Triggers can make the system more difficult to understand and debug, especially if they are not well-documented.

DML triggers can be very powerful when used correctly.

1/22/2025	10 AM - 11 AM
-----------	---------------

DDL Trigger

DDL Trigger in SQL

A **DDL (Data Definition Language)** trigger in SQL is a type of trigger that is executed in response to a DDL event, such as the creation, alteration, or deletion of database objects like tables, views, or indexes. Unlike DML triggers, which operate on data, DDL triggers operate on the structure of the database.

DDL Trigger Events:

- **CREATE:** Fired when a new database object (like a table, view, index, or stored procedure) is created.
- **ALTER:** Fired when an existing database object is modified.
- **DROP:** Fired when an existing database object is deleted.
- **RENAME:** Fired when a database object is renamed (in some databases).

Use Cases for DDL Triggers:

1. **Audit Logging:** Track changes to the schema, such as when tables or views are created, modified, or dropped.

2. **Security:** Prevent unauthorized schema changes or ensure compliance with naming conventions.
3. **Enforcing Business Rules:** Ensure that only authorized users can make certain schema modifications.
4. **Backup or Recovery:** Automatically take backups of database objects or log schema changes for disaster recovery purposes.

Syntax for Creating DDL Triggers:

```
sql
-----

CREATE TRIGGER trigger_name
AFTER | BEFORE event
ON SCHEMA
BEGIN
    -- Trigger actions (SQL statements)
END;
```

Common DDL Trigger Events:

- **AFTER CREATE:** Executes after an object is created.
- **BEFORE CREATE:** Executes before an object is created (to prevent or validate creation).
- **AFTER ALTER:** Executes after an object is modified.
- **BEFORE DROP:** Executes before an object is dropped (useful to prevent drops).
- **AFTER DROP:** Executes after an object is dropped.

Example of a DDL Trigger:

1. Audit Table Creation (AFTER CREATE Trigger):

You might want to log the creation of any new tables into an audit table.

```
sql
-----

CREATE TRIGGER log_table_creation
AFTER CREATE
```

ON SCHEMA

BEGIN

INSERT INTO ddl_audit_log (action, object_type, object_name, timestamp)

VALUES ('CREATE', 'TABLE', OBJECT_NAME, NOW());

END;

In this example:

- The trigger fires **AFTER CREATE** when a new object is created in the schema.
- It logs the CREATE action, the object type (TABLE), and the name of the created object (OBJECT_NAME) in the ddl_audit_log table.

2. Prevent Unauthorized Table Deletion (BEFORE DROP Trigger):

This example prevents the deletion of a critical table (e.g., employees).

sql

CREATE TRIGGER prevent_table_drop

BEFORE DROP

ON SCHEMA

BEGIN

IF OBJECT_NAME = 'employees' THEN

SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Deletion of employees table is not allowed';

END IF;

END;

- The trigger fires **BEFORE DROP** when an object is about to be dropped.
- If the object being dropped is the employees table, it raises an error to prevent the deletion.

3. Track Table Modifications (AFTER ALTER Trigger):

You may want to track modifications to the structure of tables, like adding or removing columns.

sql

```

-----
CREATE TRIGGER log_table_alteration
AFTER ALTER
ON SCHEMA
BEGIN
    INSERT INTO ddl_audit_log (action, object_type, object_name, timestamp)
    VALUES ('ALTER', 'TABLE', OBJECT_NAME, NOW());
END;

```

- The trigger fires **AFTER ALTER** when an object is modified in the schema.
- It logs the ALTER action along with the modified object's name.

4. Enforce Naming Conventions (BEFORE CREATE Trigger):

You might enforce a rule that ensures all table names start with a prefix like emp_.

sql

```

-----
CREATE TRIGGER enforce_table_naming_convention
BEFORE CREATE
ON SCHEMA
BEGIN
    IF OBJECT_NAME NOT LIKE 'emp_%' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Table name must start with
"emp_";
    END IF;
END;

```

- The trigger fires **BEFORE CREATE** when a table is about to be created.
- It checks the name of the object being created and raises an error if it does not start with the emp_ prefix.

DDL Trigger Events Supported in Different Databases:

- **SQL Server:** SQL Server supports DDL triggers like CREATE, ALTER, DROP events, and you can use them to track schema changes.

- **Oracle:** Oracle provides **DDL Triggers** that can respond to DDL events such as CREATE, ALTER, and DROP for tables, views, and other objects.
- **MySQL:** MySQL doesn't natively support DDL triggers in the same way as SQL Server and Oracle, but you can use events or workarounds like logging.

Example DDL Trigger in SQL Server:

In SQL Server, you can create a DDL trigger like this:

sql

```
CREATE TRIGGER ddl_trigger_example
ON DATABASE
FOR CREATE_TABLE, DROP_TABLE, ALTER_TABLE
AS
BEGIN
    PRINT 'A table has been created, dropped, or altered';
    -- Optionally log the event to a logging table
    INSERT INTO ddl_log (action, table_name, timestamp)
    VALUES (EVENTDATA().value('/EVENT_INSTANCE/EventType)[1]', 'nvarchar(100)'),
            EVENTDATA().value('/EVENT_INSTANCE/ObjectName)[1]', 'nvarchar(100)'),
            GETDATE());
END;
```

- **ON DATABASE:** The trigger is set at the database level.
- **FOR CREATE_TABLE, DROP_TABLE, ALTER_TABLE:** The trigger fires for table creation, modification, or deletion events.
- **EVENTDATA():** A function that provides XML data about the event, which you can parse to extract relevant information.

Advantages of DDL Triggers:

1. **Security:** Prevent unauthorized changes to the schema, such as dropping critical tables or views.
2. **Auditing:** Track who made changes to the schema and when, providing a history of schema modifications.

3. **Automation:** Automate tasks like logging or backup when a schema object is created, modified, or dropped.

Disadvantages of DDL Triggers:

1. **Performance Overhead:** DDL triggers can add overhead to schema changes, especially in environments with frequent DDL operations.
2. **Complexity:** Overuse of DDL triggers can complicate debugging and maintenance, as schema changes might be delayed or blocked by triggers.
3. **Limited Support:** Some databases (like MySQL) don't support DDL triggers natively, so workarounds might be required.

1/22/2025	11 AM - 12 PM	PIVOT
-----------	---------------	-------

PIVOT in SQL

If you meant **PIVOT**, then in SQL, **PIVOT** is used to transform or rotate rows into columns. It is typically used when you need to convert data from a long format (many rows) into a wide format (fewer rows with multiple columns).

Here's an example of how the PIVOT operation works in SQL:

Example: Using PIVOT to summarize sales data by product and month

Suppose you have the following sales data in a table called Sales:

Product	Month	Sales
---------	-------	-------

Product A	January	100
-----------	---------	-----

Product A	February	200
-----------	----------	-----

Product B	January	150
-----------	---------	-----

Product B	February	250
-----------	----------	-----

You want to pivot this data to show each product's sales for January and February in separate columns:

```
sql
```

```
-----
```

```
SELECT *
```

```
FROM
```

(SELECT Product, Month, Sales

FROM Sales) AS SourceTable

PIVOT

(SUM(Sales) FOR Month IN ([January], [February])) AS PivotTable;

This would result in:

Product January February

Product A 100 200

Product B 150 250

1/22/2025	2 PM - 3 PM	SQL Functions
-----------	-------------	---------------

In SQL, a **function** is a stored program that can take input values (arguments), process them, and return a result. Functions in SQL can be used to perform calculations, manipulate data, or even return a set of rows or values based on the logic defined within the function.

There are two types of functions in SQL:

1. **Built-in SQL Functions:** These are pre-defined functions provided by the database system to perform common operations (e.g., string manipulation, mathematical calculations, date operations).
2. **User-defined Functions (UDFs):** These are custom functions created by users to extend the functionality of the database system based on specific requirements.

Types of Built-in SQL Functions:

1. Aggregate Functions:

These functions perform a calculation on a set of values and return a single value.

- **COUNT():** Returns the number of rows that match a specified condition.
- **SUM():** Returns the sum of a numeric column.
- **AVG():** Returns the average value of a numeric column.
- **MIN():** Returns the minimum value in a set.
- **MAX():** Returns the maximum value in a set.

Example:

sql

SELECT COUNT(*) FROM employees; -- Returns the total number of employees.

2. Scalar Functions:

These functions operate on a single value and return a single result.

- **LEN() or LENGTH():** Returns the length of a string.
- **UPPER():** Converts a string to uppercase.
- **LOWER():** Converts a string to lowercase.
- **CONCAT():** Concatenates two or more strings together.
- **ROUND():** Rounds a numeric value to a specified number of decimal places.
- **NOW():** Returns the current date and time (depends on the database).
- **ABS():** Returns the absolute value of a number.

Example:

sql

SELECT UPPER('hello'); -- Returns 'HELLO'

3. Date and Time Functions:

These functions are used to manipulate or retrieve information about date and time.

- **NOW():** Returns the current date and time.
- **CURDATE():** Returns the current date.
- **DATEADD():** Adds a specified time interval to a date (SQL Server).
- **DATEDIFF():** Returns the difference between two dates.
- **EXTRACT():** Extracts part of a date (like year, month, or day).

Example:

sql

SELECT DATEDIFF('2025-01-01', '2024-01-01'); -- Returns 365 (difference in days).

4. String Functions:

These functions are used to manipulate text data.

- **SUBSTRING():** Extracts a substring from a string.
- **TRIM():** Removes leading and trailing spaces from a string.
- **REPLACE():** Replaces occurrences of a substring with another substring.
- **CHARINDEX():** Finds the position of a substring within a string.
- **LEFT():** Returns a specified number of characters from the left side of a string.

Example:

sql

SELECT REPLACE('hello world', 'world', 'everyone'); -- Returns 'hello everyone'

User-defined Functions (UDFs):

User-defined functions (UDFs) are custom functions that you create to perform specific operations that are not provided by the built-in SQL functions. UDFs are created using the CREATE FUNCTION statement.

There are two types of UDFs:

1. **Scalar Functions:** Return a single value based on the input.
2. **Table-Valued Functions:** Return a table (set of rows) based on the input.

1. Scalar User-Defined Function (UDF):

Here is an example of creating a scalar function that takes a number and returns its square.

sql

```
CREATE FUNCTION dbo.SquareNumber (@Number INT)
```

```
RETURNS INT
```

```
AS
```

```
BEGIN
```

```
    RETURN @Number * @Number;
```

```
END;
```

You can use this function in a query as follows:

sql

```
SELECT dbo.SquareNumber(5); -- Returns 25
```

2. Table-Valued Function (UDF):

Here is an example of creating a table-valued function that returns a table with all employees in a given department.

sql

```
CREATE FUNCTION dbo.GetEmployeesByDepartment (@DeptID INT)
```

```
RETURNS TABLE
```

```
AS
```

```
RETURN (
```

```
    SELECT EmployeeID, Name, Department
```

```
    FROM Employees
```

```
    WHERE DepartmentID = @DeptID
```

```
);
```

You can use this function in a query like this:

sql

`SELECT * FROM dbo.GetEmployeesByDepartment(1);`

SQL Function Example Summary:

1. Built-in Functions:

- **UPPER():** Converts a string to uppercase.
- **COUNT():** Counts the number of rows.
- **NOW():** Gets the current date and time.
- **LEN():** Gets the length of a string.

2. User-Defined Functions (UDFs):

- **Scalar Function:** Returns a single value (e.g., square of a number).
- **Table-Valued Function:** Returns a table (e.g., list of employees in a department).
- **Multi-Table Valued Function:**

Benefits of SQL Functions:

- **Reusability:** Functions can be used multiple times in queries, improving code reuse.
- **Abstraction:** Complex logic can be encapsulated in functions, making queries simpler and easier to understand.
- **Modularity:** You can break down complex calculations into smaller, manageable pieces.

Conclusion:

SQL functions are essential for performing operations on data, manipulating strings, dates, or numbers, and allowing you to extend the capabilities of SQL by creating your own custom functions. Built-in functions provide essential tools for data manipulation, and user-defined functions give you the flexibility to create your own solutions for complex data operations.